AI이노베이션 중간고사 복습자료

1. LLM의 정의와 작동 원리

1.1 LLM이란

LLM(Large Language Model)은 인터넷 규모의 방대한 텍스트 데이터를 학습한 거대 신경망이다. 세 단어의 의미는 다음과 같다.

- **Large**: 수십억~수조 단어(Billions/Trillions of words)의 데이터를 활용하고, 수십억~수천억 개의 파라미터를 가진다.
- **Language**: 인간 언어(Human language)의 문법, 주제, 상식적 추론 패턴을 학습한다.
- **Model**: 규칙 기반(Rule-based)이 아닌, 신경망(Neural Networks) 기반의 확률적 시스템이다.

1.2 작동 원리: 확률적 예측 기계

LLM의 핵심 메커니즘은 **다음 단어 예측(Next Token Prediction)** 이다. 이전 단어들의 시퀀스를 기반으로 다음 단어의 확률을 예측하여 텍스트를 생성한다.

예: "The cat sat on the" 다음에 올 단어의 확률:

- mat: 85%
- floor: 10%
- dog: 5%

이 확률은 방대한 텍스트에서 학습한 **단어 간의 확률적 관계(Probabilistic Relationships)** 에 기반한다.

1.3 Transformer 아키텍처

LLM의 핵심 딥러닝 구조는 **Transformer**다. GPT는 "Generative Pre-trained Transformer"의 약자다.

- 입력 시퀀스를 받아 Transformer 네트워크 블록을 통과시키고, 출력 확률을 생성한다.
- **Attention Mechanism**이 핵심 기능이다: 시퀀스 내 관련 있는 부분에 선택적으로 집중(Selectively focus)하여 복잡한 패턴을 포착한다.
- **장거리 의존성(Long-range dependencies)** 처리가 가능하다: 문장이나 문단에서 멀리 떨어져 있는 단어 간의 관계를 파악한다.

1.4 창발적 능력 (Emergent Abilities)

모델 크기와 데이터가 증가함에 따라 단순 생성을 넘어선 **일반화(Generalization) 능력**이 획득된다.

- **In-context Learning**: 가중치 업데이트 없이 프롬프트 내 소수의 예시만으로 새로운 태스크를 수행하는 능력이 발현된다.
- **Intelligent Agents로의 진화**: 단순한 텍스트 생성을 넘어, 환경을 인지하고, 도구를 사용하여 복잡한 문제를 해결하며, 의사결정(Decision-making)을 내리는 에이전트로 발전하고 있다.

1.5 LLM의 7가지 핵심 한계

번호	한계	설명
1	Static Data (정적 데이터)	학습 시점(Cut-off) 이후의 최신 정보를 알 수 없다
2	Falsehoods (거짓 생성)	사실이 아닌 내용을 그럴듯하게 생성하는 환각 (Hallucination) 현상
3	Reasoning (추론 실패)	논리적 추론 능력의 근본적 한계
4	Context Window (문맥 한계)	제한된 입력 길이를 초과할 경우 단기 기억 상실 증상 발생
5	Transparency (블랙박스)	거대 신경망의 복잡성으로 인한 해석 가능성 (Interpretability) 결여
6	Computing Requirements	거대한 스케일로 인해 훈련 및 추론 단계에서 막대한 비용 발생
7	Model Collapse (데이터 오염)	AI 생성 데이터가 새로운 모델 훈련에 유입되며 성능 점진적 저하

핵심 구조적 취약점 정리:

범주	요구사항	Vanilla LLM의 현실
Knowledge (지식)	최신성, 사실성	정적 데이터, 환각 발생
Cognition (인지)	논리적 추론	확률적 단어 조합에 불과
System (시스템)	투명성, 효율성	블랙박스 구조, 막대한 환경 비용

---

2. RAG 아키텍처

2.1 RAG란

RAG(Retrieval-Augmented Generation)는 **검색 모델(Retrieval models)과 생성 모델(Generative models)의 유기적 결합**이다. Meta가 2020년에 제안했다.

LLM의 한계를 극복하는 핵심 전략으로, 신뢰할 수 있는 최신 외부 데이터(External data)를 실시간으로 주입하여 사실 정확성, 논리적 추론, 문맥 관련성을 획기적으로 향상시킨다.

2.2 RAG의 작동 구조 (4단계)

1. **Proprietary Knowledge**: 외부 기업 데이터 및 최신 문서를 저장/인덱싱한다.
2. **Retrieved Context**: 사용자 Query에 기반한 관련 정보를 검색 및 추출한다.
3. **Context Enhanced Prompt**: 검색된 문맥을 Prompt에 병합하여 LLM에 전달한다.
4. **AI Response**: 고유 지식 기반의 정확하고 문맥이 풍부한 응답을 생성한다.

2.3 RAG가 극복하는 LLM 한계

한계	RAG의 해결 방식
Fact Retention (사실 유지)	외부 정보를 직접 참조하여 환각 억제
Improved Reasoning (추론 향상)	질문과 직결된 문서를 제공함으로써 맥락 확보
Context Relevance (문맥 적합성)	정적 데이터에 의존하지 않는 실시간/최신 정보 반영
Validation & Trust (검증 및 신뢰)	출처(Source)를 명시하여 투명성 제공

2.4 패러다임 전환: LLM을 운영체제로

- LLM은 단순한 지식 창고가 아닌, 언어적 자원을 관리하는 **운영체제(Operating System)** 다.
- Prompt 작성은 저수준 제어에 불과한 **어셈블리어(Assembly Language)** 수준이다.
- RAG는 검색(Retrieval)과 생성(Generative)을 결합한 **고수준 지식 집약적 아키텍처**다.

2.5 RAG의 주의사항

- RAG는 **만병통치약(Silver bullet)이 아니다**.
- 부정확한 문서가 검색되거나 LLM이 오독할 경우 여전히 오류가 발생할 수 있다.
- 지속적인 교차 검증(Double-check) 및 시스템 Evaluation 프로세스가 필수다.
- Fine-tuning과 RAG는 경쟁이 아닌 **상호보완적(Complementary) 기술**이다.

2.6 RAG vs Fine-tuning vs Prompt Engineering 비교

비교 기준	RAG	LoRA	Full Fine-tuning
비용(Cost)	낮음 ($)	중간 ($$)	매우 높음 ($$$)
모델 변형	없음 (In-Prompt)	일시적/모듈화	영구적 (Weights)

비교 기준	RAG	LoRA	Full Fine-tuning
데이터 최신화	실시간 (즉시 반영)	재학습 필요 (중간)	재학습 필요 (매우 느림)

Fine-tuning의 본질: 범용 데이터로 학습된 기본(Pre-trained) 모델에 특정 도메인 데이터를 추가 학습하여 영구적으로 가중치를 변형하는 것이다.

LoRA(Low-Rank Adaptation): 기존 신경망 가중치를 동결(Frozen)하고, 두 개의 작은 행렬(Low-rank matrices)만 학습하여 동작을 변경하는 효율적 대안이다. 다만 여전히 컴퓨팅 리소스 및 전문성이 요구된다.

---

3. LlamaIndex 프레임워크

3.1 LlamaIndex란

사용자 고유 지식(Custom Knowledge)과 LLM의 능력을 연결하는 프레임워크다. 2022년 Jerry Liu가 개발했다. 실제 데이터 기반의 정확하고 관련성 높은 응답을 도출한다.

3.2 설계 철학: Progressive Disclosure of Complexity

"점진적인 복잡성 노출" — 초보자부터 전문가까지 모두를 만족시키는 아키텍처.

- **Surface Level**: 단 몇 줄의 코드만으로 외부 데이터 연결 및 LLM 쿼리 완료.
- **Deep Level**: 필요에 따라 특화된 인덱스 구조, 쿼리 알고리즘, 세밀한 매개변수를 직접 제어 가능.

3.3 기본 워크플로우 (3단계)

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

documents = SimpleDirectoryReader('files').load_data()     # Step 1: Data Loading
index = VectorStoreIndex.from_documents(documents)          # Step 2: Indexing
query_engine = index.as_query_engine()                      # Step 3: Querying
response = query_engine.query("summarize each document")
```

3.4 LlamaIndex v0.10 모듈형 아키텍처

패키지	역할
`llama-index-core`	프레임워크의 근간. 핵심 기능만 포함하여 가벼운 설치 보장
`llama-index-integrations`	300개 이상의 확장 애드온 (Custom LLMs, Data Loaders, Vector Stores)

패키지	역할
`llama-index-packs`	개발자 커뮤니티 제공 50개 이상의 레디메이드 애플리케이션 템플릿
`llama-index-cli`	CLI 도구

Integration 패키지는 개별 설치가 필수다. 예: `pip install llama-index-llms-mistralai`

3.5 핵심 활용 사례 4가지

1. **문서 컬렉션 검색 엔진**: PDF, Word, 사내 위키 등 모든 문서를 인덱싱하여 문맥 기반의 강력한 맞춤형 정보 검색.
2. **사내 맞춤형 챗봇**: 기업 고유 정책, 전문 용어, 지침을 완벽히 숙지한 인간 전문가 수준의 고객 서비스 및 사내 헬프데스크 AI 구축.
3. **대규모 보고서 요약**: 방대한 논문이나 보고서에서 가장 중요한 핵심 포인트만 간결하게 추출 및 요약.
4. **스마트 어시스턴트/데이터 에이전트**: 조직 고유의 복잡한 다단계 작업의 뉘앙스를 학습하고, 단순 답변을 넘어 인사이트와 실행 가이드를 제공하는 에이전트.

---

4. 핵심 데이터 구조: Document, Node, Index

4.1 세 구성요소의 역할

구성 요소	역할	단계	데이터 단위
Document	원시 데이터의 표준화된 컨테이너	Ingest (수집)	Macro (원시 데이터)
Node	처리 가능한 작은 텍스트 청크	Structure (구조화)	Micro (텍스트 조각)
Index	검색 최적화를 위한 구조체	Query (검색)	System (데이터베이스)

전체 흐름: **Raw Data → Document → Node → Index → Query Engine → Response**

4.2 Document 상세

정의: 다양한 데이터 소스(텍스트, 파일 등)를 LlamaIndex가 이해할 수 있는 표준 객체로 변환한 기본 단위.

객체 구조:

- `text` : 문서의 실제 텍스트 내용
- `metadata` : 사전(Dictionary) 형태의 추가 정보 (검색 정밀도를 높이는 부가 데이터)
- `id_` : 자동 또는 수동으로 할당되는 고유 식별자(UUID)

데이터 소스의 유연성: 단순 파일(PDF, 텍스트)을 넘어 데이터베이스, API 응답 등 모든 형태의 원시 데이터를 수용 가능. LlamaHub의 130개 이상의 사전 정의된 Data Loader(Connector)를 통해 대규모 데이터를 자동 수집하고 Document 객체로 일괄 변환한다.

4.3 Node 상세

정의: Document를 처리 가능한 작은 텍스트 청크(Chunks)로 분할한 핵심 단위.

Node가 필요한 이유:

1. **LLM 프롬프트 한계 극복**: 방대한 문서를 Context Window 제약에 맞게 분할.
2. **의미론적 단위 생성**: 특정 정보에 집중된 작은 단위로 데이터를 조직화하여, 검색과 분석의 정확도 및 효율성을 극대화.
3. **네트워크 및 관계 형성**: 파편화된 고립 데이터가 아닌, 노드 간 상호 연결된 지식망(Knowledge Graph)을 구축하기 위한 기초 물리적 단위 제공.

Node 추출 메커니즘 (TokenTextSplitter):

- `chunk_size`: 한 번에 처리할 최대 토큰 크기
- `chunk_overlap`: 맥락 유지를 위한 노드 간 겹침 구간
- 주의: metadata가 너무 방대하면 chunk_size 내에서 실제 텍스트가 들어갈 공간이 부족해져 검색 효율이 저하될 수 있다.

Node 간 관계(Relationships):

- **PREVIOUS/NEXT**: 원본 Document 내에서의 물리적 순서를 추적하는 순차적 관계. 자동으로 생성된다.
- **PARENT**: 계층 구조에서 상위 수준의 노드 (예: 섹션-단락 관계)
- **CHILD**: 부모 노드에 종속된 하위 및 세부 노드
- **SOURCE**: 노드의 출처가 되는 원본 Document 추적 (Provenance 제공)

metadata 상속: 하위 Node들은 원본 Document가 가진 metadata를 자동으로 상속받아 출처 정보를 유지한다.

4.4 Index 상세

정의: 흩어진 Node들을 특정 수학적/논리적 기준에 따라 체계적으로 배치하여 저장 및 검색(Retrieval) 효율성을 극대화하는 데이터 구조.

비유: 옷장 서랍을 종류별로 정리해두면 원하는 양말을 찾기 위해 전체를 뒤지지 않고 정확한 포켓에서 즉시 추출할 수 있는 것과 같다.

Index 유형 비교:

Index 이름	구조 및 작동 원리	최적 활용처	빌드 시 LLM 의존도	비용
SummaryIndex	노드를 순차적인 리스트 형태로 연결	전체 문서 정독 및 맥락 전체 요약	없음	매우 낮음
VectorStoreIndex	텍스트를 임베딩 벡터로 변환하여 수학적 유사도로 그룹화	대부분의 표준 RAG 검색	없음 (Embedding만)	낮음
TreeIndex	노드를 계층적 트리 구조로 배치, 부모가 자식 요약	대규모 문서의 구조적 요약 검색	계층별 요약 시 대량 사용	매우 높음
KeywordTableIndex	중요 키워드와 해당 노드를 1:1 매핑	특정 성분이나 명확한 키워드 기반 정보 탐색	추출 시 사용 (RAKE 대체 가능)	낮음
DocumentSummaryIndex	원본 문서별 전체 요약본 생성 후 내부 node와 1:1 매핑	방대한 문서 저장소 환경	요약 생성 시 사용	보통
KnowledgeGraphIndex	추출된 triplet(주어-술어-목적어) 기반 지식 그래프 구축	복잡한 엔티티 관계 분석	구조 추출 시 필수	높음

4.5 Query Engine

Query Engine은 사용자의 질의를 처리하는 파이프라인으로, 내부적으로 3단계로 작동한다.

1. **Retriever (검색기)**: 사용자의 질문을 분석하여 Index에서 가장 관련성 높은 Node들을 추출 (Fetch & Rank).
2. **Node Postprocessor (후처리기)**: 추출된 노드들을 재정렬(Re-ranking)하거나 불필요한 노드를 필터링하여 노이즈를 사전 제거.
3. **Response Synthesizer (응답 합성기)**: 최종 정제된 노드의 컨텍스트와 사용자의 질문을 결합하여 프롬프트를 구성, LLM에 전달하여 최종 자연어 응답을 생성.

---

5. LLM 커스터마이징

5.1 Settings 클래스

LlamaIndex의 `Settings`는 단순한 변수가 아닌, 인덱싱과 쿼리 전반에 걸쳐 공유되는 **글로벌 컨텍스트**다. 한 번의 정의로 모든 후속 LlamaIndex 오퍼레이션에 동일한 구성(Configuration)이 적용된다.

```
from llama_index.llms.openai import OpenAI
from llama_index.core.settings import Settings
```

```
Settings.llm = OpenAI(temperature=0.8, model="gpt-4")
```

5.2 Temperature

LLM 출력의 무작위성을 제어하는 파라미터다. 다음 토큰 예측의 확률 분포 평탄화를 통해 무작위성을 제어한다.

Temperature	특성	용도
0 (또는 매우 낮음)	Focused, Deterministic	코드 생성, 데이터 분석 등 사실 기반 작업
0.5+	Random, Creative	글쓰기, 챗봇 등 창의적 작업

주의: 완벽한 0이라도 부동소수점 한계 등으로 미세한 변동 가능성이 존재한다.

5.3 LLM 선택 매트릭스

모델	속도	비용	최적 용도
GPT-3.5-Turbo	최상 (Fastest)	최저 (Lowest)	기본 Q&A, 범용 RAG
GPT-4	보통 (Moderate)	높음 (High)	복잡한 추론, 최고 성능 요구
Claude-2	보통 (Moderate)	보통 (Moderate)	매우 긴 컨텍스트 처리

---

6. Data Ingestion (데이터 수집)

6.1 Garbage In, Garbage Out

RAG 시스템의 최종 성능은 **데이터 품질이 결정**한다. 중복(Duplicates), 오류(Errors), 모호함(Ambiguous), 편향성(Biased), 구형(Outdated) 데이터는 RAG 시스템 혼란 및 환각을 유발한다.

자문 기준: **"이 데이터를 수동으로 검색한다면, 원하는 정보를 쉽게 찾을 수 있는가?"**

6.2 데이터의 동적 변화

조직의 지식 저장소는 결코 정적(Static)이지 않다. Production 레벨의 RAG Application은 정기적 리뷰, 지속적 정보 통합(Integration), 구형 데이터 삭제를 위한 체계적(Systematic) 구축이 요구된다.

6.3 LlamaHub

정의: 180개 이상의 Data Reader(Connector)를 보유한 방대한 Integration 라이브러리.

핵심 가치: 개발자가 개별 Parser를 처음부터 작성할 필요를 제거(바퀴의 재발명 방지). 외부 데이터를 추출하여 표준 LlamaIndex Document 객체로 자동 변환한다.

모듈 아키텍처: 핵심(Core) 프레임워크와 확장(Integration) 패키지가 완벽하게 분리되어 있다. 경량화 및 의존성(Dependency) 최소화를 위해, 특정 Data Loader 사용 전 해당 Integration 패키지를 개별 설치해야 한다.

6.4 주요 Ingestion 방법

1) 개별 소스 (Web & Database):

- `SimpleWebPageReader`: URL 기반 웹페이지 텍스트 추출. `html_to_text=True` 설정 시 html2text 의존성이 필요하다. `metadata_fn`으로 사용자 정의 메타데이터(예: 타임스탬프)를 부여하여 시계열 트래킹이 가능하다.
- `DatabaseReader`: SQL 쿼리 실행 결과를 Document 객체로 변환. 각 쿼리 결과 Row가 단일 Document로 매핑된다.

2) 대량 데이터 (Bulk Loading):

- `SimpleDirectoryReader`: 대량 데이터 Ingestion을 위한 범용 도구. 확장자 기반 자동 인식(PDF, DOCX, CSV 등), `recursive=True`로 하위 디렉토리까지 일괄 탐색 및 로드, 바이너리 파일 내부 처리 (예: PDF→PyPDF, 이미지→Pillow).

3) 고급 멀티모달 파싱 (LlamaParse):

- 복잡한 파일 구조(표, 이미지, 수식, 코드 스니펫)를 위한 LLM 기반 호스팅 서비스.
- `parsing_instruction` 파라미터를 통해 추출 방식을 자연어로 제어한다 (예: "각주 제외, 표 마킹, 이미지 추출").
- Rich Structured Output: JSON output mode 지원.
- SimpleDirectoryReader와 연동하여 디렉토리 단위의 일괄 고품질 Ingestion을 지원한다.

---

7. Parsing: Document를 Node로 변환

7.1 왜 Parsing이 필요한가

- 거대한 Document를 LLM이 처리하기 쉬운 세분화된(Granular) 단위로 분할하여, 고품질 검색(Retrieval)을 위한 해상도(Resolution) 확보.
- LLM의 Context Window 한계 극복 및 연산 효율성 극대화.
- 원본 문서의 내부 구조(Internal structure)를 정확히 반영하여 유지.

7.2 Simple Text Splitters

원시 텍스트(Raw text) 분할에 최적화된 Flat structure 분할 도구.

Splitter	작동 방식
SentenceSplitter	문장 경계(Boundary)를 유지하며 분할
TokenTextSplitter	Token 단위 분할 (주요 파라미터: separator, backup_separators)
CodeSplitter	AST(Abstract Syntax Tree) 기반 분할. 프로그래밍 언어의 논리적 흐름과 함수 구조 유지

7.3 Advanced Node Parsers

파일 구조, 시맨틱 문맥, 계층적 관계를 인지하는 지능형 프로세싱.

1) SentenceWindowNodeParser (Contextual Window):

- 개별 문장 단위로 분할하되, 주변 문장(Window)을 해당 Node의 metadata에 포함.
- `window_size`: 타겟 문장 좌우로 포함할 문장의 수.
- 검색 시 타겟 문장만 매칭하되, LLM에는 풍부한 주변 문맥(Context)을 제공하여 답변 품질 향상.

2) Format-Specific Parsers:

- `HTMLNodeParser`: BeautifulSoup 기반. 지정된 HTML 태그(예: `<p>`, `<span>`) 기준으로 텍스트 추출 및 병합.
- `MarkdownNodeParser`: 마크다운 헤더 계층을 반영하여 분할 및 metadata 병합.
- `JSONNodeParser`: JSON의 구조화된 데이터 계층 분할.
- Tip: `SimpleFileNodeParser`를 사용하면 확장자에 따라 위 Parser들을 자동 매핑 적용.

3) HierarchicalNodeParser (Relational Parser):

- 동일 문서를 여러 크기(Level)로 분할하여 부모-자식 관계의 트리 Node를 생성.
- `chunk_sizes` 파라미터 (예: [2048, 512, 128]).
- 검색 시 좁은 문맥(Detail)과 넓은 문맥(Summary)을 유연하게 조정 가능.

4) UnstructuredElementNodeParser:

- 텍스트와 표(Table) 등 이질적 요소를 분리하여 관계 매핑.

7.4 핵심 파라미터

chunk_size (LlamaIndex Default: 1024):

- Node 당 허용되는 최대 Token 수.

- Too Small → 중요 문맥 유실, LLM 답변 품질 저하.
- Too Large → Prompt 사이즈 증가, 연산 비용 상승, 생성 속도 지연(Latency).
- RAG 성능 최적화를 위한 **실험적(Experimental) 크기 조정 필수**.

chunk_overlap (LlamaIndex Default: 20):

- 인접한 연속 Node 간에 공유(중복)되는 Token 수.
- 분할 경계면에서의 아이디어 연속성(Continuity) 보장.
- LLM 처리 시 문맥 단절(Contextual break) 방지.

include_prev_next_rel (Default: True):

- 파싱 시 연속된 Node 간에 PREVIOUS/NEXT 관계를 자동 삽입.
- Cluster Retrieval: 타겟 Node 검색 시, 연결된 주변 Node 군(Cluster)을 함께 가져와 포커스된 문맥 제공.

7.5 Splitters vs Parsers 비교

구분	Text Splitters	Node Parsers
복잡도	Basic (규칙 기반)	Advanced (시맨틱/구조적)
대상 데이터	Raw Text (평면적 분할)	Formatted/Relational Docs
특징	chunk_size 의존도 높음	메타데이터를 통한 문맥 확장, 계층 구조 및 파일 포맷의 완벽한 인지

---

8. Metadata 엔지니어링

8.1 Metadata의 역할

데이터에 부가적인 단서를 결합하여 LLM의 이해도를 향상시킨다. 단순 텍스트 기반 검색을 넘어 카테고리, 작성자, 타임스탬프 기반의 정밀한 query 처리가 가능하다.

Blind node (raw text만) vs Enriched node (metadata 포함): metadata가 있으면 검색 정밀도가 획기적으로 향상된다.

8.2 Metadata 할당 메커니즘

- **수동 할당**: `Document(text="...", metadata={'author': 'John Doe'})` 처럼 직접 주입.

- **자동 할당**: `SimpleDirectoryReader`와 `file_metadata` 콜백을 활용하여 ingestion 과정에서 자동화.
- 소규모 환경에서는 수동, 대규모 환경에서는 자동 파이프라인 구축이 필수.

8.3 속성 상속의 원리

상위 Document에 부여된 metadata는 하위 분할 객체인 Node로 **자동 전파**된다. 중복 정의 방지 및 ingestion 시간이 단축된다.

8.4 Metadata Extractor 종류

Extractor 클래스	핵심 기능	활용 사례
SummaryExtractor	인접 node 요약 생성	고객 지원 문서의 과거 해결 내역 요약
QuestionsAnsweredExtractor	답변 가능한 질문 생성	FAQ 시스템 검색 최적화
TitleExtractor	제목 없는 텍스트에서 제목 추출	디지털 라이브러리 문서 분류
EntityExtractor	고유 명사(인물, 장소 등) 식별 (NER)	법률 문서 내 특정 법인 및 인물 검색
KeywordExtractor	핵심어 추출	콘텐츠 추천 엔진 정합성 향상
PydanticProgramExtractor	구조화된 스키마 기반 추출	단일 LLM 호출로 다중 필드 데이터 획득

SummaryExtractor 상세: 설정값 `summaries=['prev', 'self', 'next']`로 현재 노드뿐 아니라 이전/다음 노드의 요약을 metadata에 포함. 문서 전체를 탐색하지 않고 요약본만으로 검색 가능성을 판단하여 retrieve 단계의 효율성을 극대화한다.

QuestionsAnsweredExtractor 상세: 텍스트가 답변할 수 있는 예상 질문을 역설계하여 metadata화. 특정 질문에 대한 명시적 해답을 가진 node 도출에 최적화된다.

EntityExtractor 상세: NLTK 토크나이저와 span-marker를 활용한 Named Entity Recognition (NER) 수행. `label_entities=True`로 인물, 조직, 장소를 라벨링한다.

8.5 MetadataMode: 가시성 제어

과도한 metadata는 LLM의 프롬프트 컨텍스트를 오염시켜 인지적 편향(Bias)을 유발할 수 있다 (예: 시스템 파일명이 환각 트리거).

- `MetadataMode.ALL`: 전체 노출
- `MetadataMode.LLM`: LLM 전용
- `MetadataMode.EMBED`: Embedding 전용

- `excluded_llm_metadata_keys`를 통해 특정 속성을 LLM의 시야에서 은닉하면서 embedding 공간에만 노출 가능.

8.6 비용의 역설 (Compute Cost Paradox)

대규모 node에 대해 Summary, Keyword 추출 등 정교한 metadata를 수행할 경우, 기하급수적인 LLM 호출이 발생한다. Rich Context 확보와 API 과금 간의 치명적인 트레이드오프(Trade-off)가 존재한다.

비용 최적화 전략:

- **Batching**: Pydantic 추출기 등을 활용해 단일 호출로 다수 필드 병합 처리.
- **Cheaper Models**: 연산량이 적은 경량 모델 활용 (단, 데이터 오류 전파 리스크 고려).
- **Caching**: Ingestion pipeline을 통한 이전 추출 결과의 재사용.
- **Selective Processing**: 전체 data가 아닌 핵심 node에 대해서만 제한적 추출 수행.

8.7 MockLLM을 활용한 비용 시뮬레이션

실제 API 호출 전, 가상의 MockLLM을 로컬에 배치하여 최악의 시나리오 비용을 예측한다.

- `TokenCountingHandler`가 프롬프트 및 응답을 가로채어 정확한 token 소모량을 계산.
- 주의: 대상 LLM과 동일한 토크나이저를 설정해야 한다 (예: GPT-4의 경우 CL100K).

8.8 데이터 프라이버시

외부 클라우드 LLM 서비스로 데이터가 전송될 때 보안 규제 위반 리스크가 존재한다. 고객 민감 식별 정보, 금융 내역, 비공개 대화 이력 등이 노출되거나 유출될 가능성이 있다.

해결: NERPIINodePostprocessor:

- 데이터가 네트워크를 타기 전, 로컬 환경에서 선제적 정제 작업(Scrubbing)을 수행.
- Hugging Face의 NER 모델을 활용하여 PII를 범용 토큰으로 강제 치환.
- 예: "Dear Jane Doe, 8804 Vista Serra Dr." → "Dear [PER_5], [LOC_95] Dr."

---

9. Embedding과 VectorStoreIndex

9.1 Embedding이란

비정형 데이터(텍스트, 이미지, 오디오 등)를 기계가 이해할 수 있는 고차원 **부동소수점 숫자 배열 (Floating-point numbers)** 로 변환한 결과물이다. 모호한 언어적 뉘앙스와 시맨틱 관계를 균일한 벡터 공

간의 데이터로 구조화한다.

주의: 분할된 Chunk의 크기가 Embedding 정밀도에 직접적 영향을 미친다 (작으면 컨텍스트 손실, 크면 의미 희석).

9.2 벡터 공간 내 의미적 관계성

벡터 공간에서 **의미가 유사한 문장은 서로 근접하게 배치**된다.

예:

- "The quick brown fox jumps over the lazy dog" 와 "A fast dark-colored fox leaps above a sleepy canine" → 의미 동일, 공간적 근접
- "Apples are sweet and crunchy" → 의미 상이, 공간적 격리

9.3 VectorStoreIndex

LlamaIndex에서 가장 범용적으로 활용되는 표준 Index. 텍스트 Chunk에 대한 Embedding을 Vector Store에 저장하고, 수학적 벡터 공간에서의 Similarity Search를 통해 가장 연관성이 높은 정보를 초고속으로 Retrieval한다.

아키텍처: `from_documents()` 메서드를 통해 Document에서 Node 분할, 그리고 Embedding 매핑으로 이어지는 자동화 과정. 선택적 파라미터 `store_nodes_override` 를 통해 Vector Store 외에 원본 Node 객체의 직접 저장을 강제할 수 있다.

9.4 Top-k Similarity Search

1. 사용자의 Query를 동일한 Embedding 모델을 통해 벡터로 변환.
2. 수학적 거리를 계산하여 Vector Store에서 가장 유사한 Top-k개의 벡터를 추출.
3. 해당 벡터에 연결된 Node의 텍스트를 LLM에 전달하여 답변 생성.

9.5 유사도 측정 방식 비교

1) Cosine Similarity (LlamaIndex Default):

- 두 벡터 사이의 **각도(Angle)** 를 측정한다.
- 1에 가까울수록 유사하다.
- 벡터의 길이(문서의 분량)에 적게 영향 받으며, 벡터의 **방향성(Orientation)에 집중**한다.
- 텍스트 분석에 최적화된다.

2) Dot Product:

- 두 벡터의 대응하는 요소를 곱한 후 합산한 단일 스칼라 값.
- 방향(Direction)뿐만 아니라 **길이(Magnitude)에도 민감하게 반응**한다.
- 주의: RAG 적용 시 긴 문서가 본질적으로 큰 값을 가져, 관련성보다 문서 길이에 의해 검색 결과가 편향될 위험이 존재한다.

3) Euclidean Distance:

- 각도나 정렬 상태가 아닌, 두 벡터 끝점 사이의 **실제 직선 거리(Actual distance)** 를 측정한다.
- 실제 벡터의 차원(Dimensions)이 물리적 크기나 측정값을 의미하는 데이터 세트에서 특히 유용하다.

지표	측정 기준	길이 민감도	RAG 적합성
Cosine Similarity	벡터 간의 각도	낮음 (정규화됨)	**최상 (Default)**
Dot Product	스칼라 곱 (크기 × 방향)	매우 높음	주의 요망 (긴 문서 편향 위험)
Euclidean Distance	벡터 끝점 간 물리적 직선 거리	높음	제한적 (물리적 의미 데이터에 적합)

9.6 Embedding 모델 선택 기준 (6가지)

1. **Quantitative Performance**: MTEB 리더보드 상의 객관적 지표
2. **Qualitative Performance**: 특정 도메인 사전학습 및 의미론적 이해도
3. **Multilingual Support**: 다국어 데이터 처리 성능 (언어적 한계 고려)
4. **Cost**: API 토큰 비용 vs 로컬 인프라 컴퓨팅 비용
5. **Latency & Throughput**: 실시간 Query 처리 속도 및 최대 입력 Chunk 크기
6. **Privacy & Availability**: 오프라인 작동 필요성 및 민감 데이터 보안 준수

Default Hosted Model: OpenAI `text-embedding-ada-002` — 품질, 성능, 비용의 우수한 밸런스.

Local Models: Hugging Face `UAE-Large-V1` 등 오픈소스 모델. 프라이버시 확보 및 API 호출 비용 제로. 단, 대규모 데이터 처리 시 로컬 컴퓨팅 자원 필요.

---

10. Storage & Persistence (영구 저장)

10.1 Persistence의 필요성

- **비용 절감**: 매 세션 문서 재처리 및 embedding 재연산에 따른 컴퓨팅 비용 방지.
- **지연 시간 최소화**: 런타임 처리 생략, 사전 연산된 데이터 로드를 통한 즉각적인 애플리케이션 구동.